# Noesis: Selective Memory Injection for Cross-Tool AI Conversation Consistency

Peizheng Du
Independent Researcher
peterdu0628@gmail.com

October 2026

**Abstract**

Users converse with multiple AI assistants daily, yet each tool's memory is isolated: a stance expressed in one tool is unknown to the others, and each conversation starts from scratch. Shared and evolving memory layers already exist; this paper studies a narrower design choice: an explicit admission rule that decides whether a captured statement is firm enough to enter the prompt at all. I present **Noesis**, a local-first memory layer that (1) extracts typed *thoughts* from conversations across tools into a shared per-user store, (2) gates injection behind a confidence lifecycle in which thoughts mature from *tentative* to *settled* based on repetition, assertion strength, cross-tool corroboration, and contradiction signals, and (3) caps injected context with a token budget. Across three models (Gemini Flash-Lite, gemma3:4b, qwen2.5:3b) on 110 paired questions each, memory injection raises the keyword-hit rate on synthetic personal questions from 41.8% to 85.5%, 34.5% to 82.7%, and 37.3% to 62.7% respectively (+43.6, +48.2, and +25.5 percentage points; all $p < 0.001$, McNemar's test). At 500 stored memories, Noesis keeps 9 of 10 targets at rank 1 with about 207 injected tokens, 35 times fewer than injecting every memory (MRR 0.90 vs. 0.002); a standard top-5 RAG ranks as well with fewer tokens, so Noesis's distinct gain is filtering: where ungated retrieval surfaced tentative or hedged statements 19 and 13 times in two constructed tests, Noesis surfaced none; the gate filters by how firmly something was said, not by whether it is true. On strictly cross-tool queries, Noesis retrieves 37% where per-tool isolation is structurally bound to 0%. I release the system, the experiment harness, and the saved raw results.

**Keywords:** large language models; LLM memory; retrieval-augmented generation; personalization; cross-tool memory; confidence gating; local-first software

## 1 Introduction

Heavy users of AI assistants switch between ChatGPT, Claude, Gemini, and local models throughout the day. Each tool maintains its own isolated memory: a database preference carefully explained to Claude must be re-explained to GPT; a design decision reached with Gemini is invisible to Claude. The reasoning a user performs across tools, such as their positions, preferences, and open questions, evaporates at every tool boundary.

Existing systems already address much of this problem (§2). Mem0 [3] extracts and consolidates salient facts from conversations and offers a graph-based variant; OpenMemory MCP [17] shares one local store across MCP clients; Zep [21] records when facts become valid and invalid in a temporal knowledge graph; and Hindsight [12] separates facts from opinions and attaches an evolving confidence score to each opinion. *Retrieval-augmented generation* (RAG) [13] supplies the

underlying mechanism of fetching relevant context per query; its simplest variant, injecting every stored item, grows linearly with memory size and buries the relevant signal (I measure 7,192 tokens at $n$=500).

Noesis is built on a different premise: *what* gets injected matters as much as *whether* anything is injected. A captured statement should reach the prompt only when the evidence that the user holds it is strong enough: firm wording, repetition, agreement across tools, and no contradiction. This is a rule about admission, not a check on truth.

This paper makes the following contributions:

1. **A confidence lifecycle for conversational memory** (§3). Thoughts are typed (identity, preference, position, question, synthesis, event, contradiction) and mature tentative→provisional→settled via a four-signal scorer, with type-specific time decay and cluster-aware supersession that reduced stale-stance leakage from 93% to 7% (§3.3).

2. **Gated injection that scales** (§5.2–§5.3). At 500 memories Noesis injects ∼207 tokens (35 times fewer than injecting everything) while holding MRR at 0.90; a standard top-5 RAG matches that ranking at lower cost, and Noesis's gate is what keeps tentative and hedged statements out (0 vs. 19 and 0 vs. 13 in constructed tests); it does not verify facts or stop deliberate poisoning.

3. **Quantified cross-tool memory sharing** (§5.4). With a shared per-user store, Noesis retrieves 37% of strictly cross-tool queries versus 0% for per-tool isolation (50% vs. 7.5% over all 40 scenarios)—a gap that is architecturally impossible for isolated systems to close.

4. **An A/B result across three model tiers** (§5.1). On identical $n$=110 question sets, memory injection significantly raises the keyword-hit rate for a cloud model (Gemini Flash-Lite, +43.6pp), a mid-tier local model (gemma3:4b, +48.2pp), and a small local model (qwen2.5:3b, +25.5pp); all $p < 0.001$.

Storage and retrieval are local: embeddings and retrieval run on-device (sqlite-vec + all-MiniLM-L6-v2). Conversation text leaves the machine only when optional LLM-based extraction is enabled or when the proxy forwards a request to a cloud model, which then also receives the injected memories. All code, the experiment harness, the saved raw results (§4 lists which records were not kept), and a script that recomputes every reported number from them are released.[1]

## 2 Related Work

This is a targeted review, not a systematic one; sources were last checked on 17 October 2026. I cite peer-reviewed versions where they exist; preprints and vendor announcements are marked as such in the references, and statements about a system describe the cited version, not later releases.

**Foundations of agent memory.** Maintaining information beyond a context window is not new. MemGPT [19] treats context management as movement between memory tiers; Generative Agents [20] store experience, synthesize reflections from it, and rank memories by recency, importance, and relevance; and MemoryBank [28] lets memories fade along an Ebbinghaus forgetting curve, the closest precedent for Noesis's type-specific decay. Together these separate storage, consolidation, retrieval, and control. Recent surveys organize the field by the forms, functions, and

[1] https://github.com/AngryKiwiWithLegs/Noesis

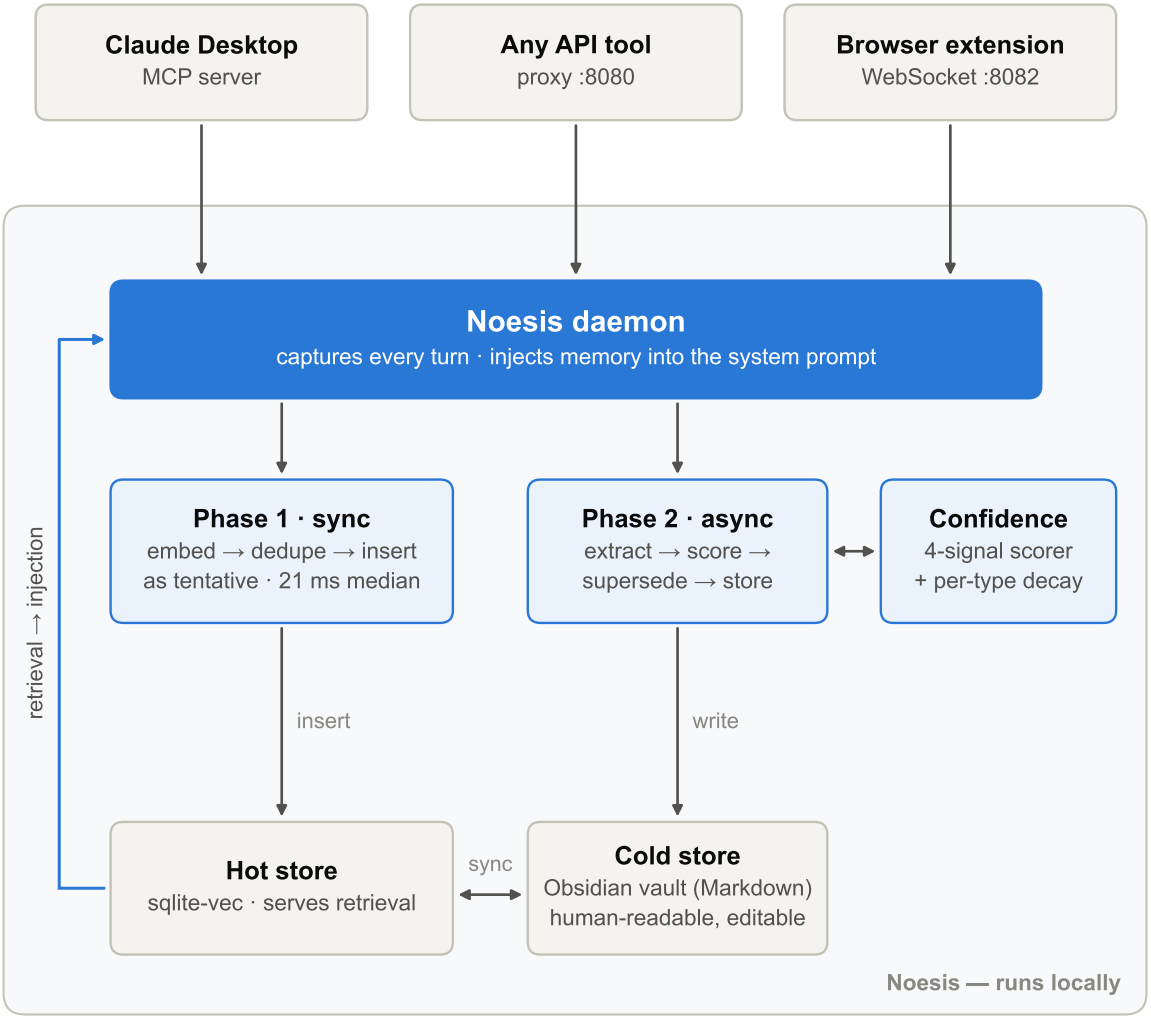



Figure 1: Noesis system architecture. Tools connect through an MCP server, an OpenAI-compatible proxy, or a WebSocket; Phase 1 stores the utterance synchronously (median 21 ms) and Phase 2 asynchronously extracts, scores, and links the thought. Hot store (sqlite-vec) serves retrieval; cold store (Obsidian vault) keeps memory human-readable and editable.

dynamics of memory [8] and, for graph-based memory, by extraction, storage, retrieval, and evolution [25]. Against this background, Noesis's candidate contribution is narrow: an *admission policy* that decides which stored items are eligible to be injected at all.

**Shared and evolving memory services.** Mem0 [3] extracts and consolidates salient facts from ongoing conversations, and its graph-based variant captures relations between them. OpenMemory MCP [17] runs a Mem0-based store locally and shares it across MCP clients such as Cursor, Claude Desktop, Windsurf, and Cline, through tools that add, search, list, and delete memories. Noesis shares this local, cross-client design. It differs in two respects: it also captures clients outside MCP, through an OpenAI-compatible proxy and a browser extension; and it withholds stored items whose status is below provisional, whereas the OpenMemory announcement describes no rule that withholds memories by confidence or status. Zep [21] builds a temporal knowledge graph (Graphiti) of episodes, entities, and communities. Each fact edge carries validity timestamps, an LLM compares new facts with related existing ones, and a contradicted fact is invalidated rather than deleted; retrieval combines cosine similarity, BM25, and breadth-first graph search with rerankers. Zep's invalidation answers the question Noesis's supersession check (§3.3) answers—is this stance still current?—with a more general, LLM-based mechanism, where Noesis uses replacement verbs and a domain keyword map that are cheaper but narrower. Zep's paper describes relevance reranking but no threshold that keeps a valid fact out of the results; Noesis's gate addresses that separate question of whether a statement was asserted firmly enough to be used.

**Hindsight.** Hindsight [12] is the closest system to Noesis. It separates an agent's memory into four networks (objective world facts, the agent's own experiences, opinions, and observations that summarize entities), extracts facts with an LLM, and recalls them through four channels (semantic, BM25, graph spreading activation, and temporal) fused with reciprocal-rank fusion, reranked with a cross-encoder, and capped by a token budget. Its opinions carry a confidence score that evidence reinforces, weakens, or contradicts. Typed memory, hybrid retrieval with a budget, and evolving belief confidence are therefore all established, and I do not claim them as contributions. The differences are narrower. First, Hindsight's confidence is attached to opinions the *agent* forms during reflection, and, in the description I examined, it is not a condition for recall; Noesis scores the *user's* own statements of every type and admits nothing below provisional into the prompt. Second, the intended deployment differs: Hindsight indexes memory in PostgreSQL with pgvector and calls an LLM to extract facts, whereas Noesis runs on SQLite with a CPU embedding model and falls back to deterministic extraction without an API key. Third, Noesis captures conversations from several different AI tools into one store. I have not compared the two systems empirically, and this paper does not show that Noesis's gate improves on confidence-annotated recall.

**Learned and graph-structured memory control.** A-MEM [24] builds Zettelkasten-style notes and updates their links and attributes as new memories arrive. GraphRAG [5] builds an entity graph with precomputed community summaries to answer global questions about a document corpus, a different task from tracking one user's changing preferences. AgeMem [27] exposes storing, retrieving, updating, summarizing, and discarding as tool actions and trains the agent's policy to use them with reinforcement learning; MRAgent [9] accesses an associative Cue–Tag–Content graph iteratively, guided by intermediate reasoning. These systems treat memory management as a control and learning problem. Noesis instead uses fixed rules: its intended advantages are transparency (each admission decision traces to a score and a threshold, and memory is human-editable Markdown), low overhead, and portability across tools, not task performance, which this paper

does not compare against them.

**Benchmarks.** LoCoMo [15] and LongMemEval [23], which tests extraction, multi-session and temporal reasoning, knowledge updates, and abstention, are the standard long-term-memory evaluations; Hindsight and MRAgent report results on both. This paper does not yet use them, and results across systems on them are comparable only when the backbone model, scoring, available history, and budget are aligned.

**Memory security.** Persistent memory adds attack surface. MINJA [4] plants malicious records in an agent's memory through ordinary queries and observed outputs, without direct access to the memory store. A recent survey [14] organizes attacks and defenses across six lifecycle phases (write, store, retrieve, execute, share and propagate, forget and roll back) and argues that memory security cannot be retrofitted at retrieval or execution time alone. Noesis's confidence gate acts at retrieval time and filters by how firmly something was said, so it does not address write-time attacks: a firmly worded planted statement reaches provisional (§3.2, §9). A shared cross-tool store also adds a propagation path, since a record written through one tool is injected into the others. "Local-first" here describes storage and retrieval; when configured, conversation text still reaches a cloud extraction model, and injected memories reach whichever provider answers the request.

**Retrieval-augmented generation.** RAG [13] and successors retrieve per-query context. Self-RAG [2] and FLARE [10] improve retrieval quality or adaptivity, but address corpus QA, not a user's evolving personal memory. My scale experiment (§5.3) shows that injecting every stored memory fails for personal memory (cost grows linearly and ranking collapses) and that a standard top-5 retriever avoids both. Noesis pairs hybrid retrieval (BM25 + vector + RRF, with time-decay weighting) with a hard token budget and a confidence gate.

**Personalization.** PALR [26] and related work personalize outputs from user interaction history, typically for recommendation-style tasks. Chat-tool vendor memory features (e.g., ChatGPT memory) are per-product and closed. Noesis's cross-tool experiment (§5.4) shows the benefit of user-scoped over tool-scoped access; it does not compare Noesis with other shared stores.

**Visualization of embedding spaces.** t-SNE [22] and UMAP [16] project high-dimensional embeddings for inspection; force-directed layouts [6] model attraction/repulsion in 2D. My star map (§6) extends force-directed layout to 3D and adds weak axis springs, so semantic clusters still form freely while time reads left to right and confidence bottom to top.

## 3 System Design

Noesis sits between AI tools and model providers (Figure 1). API clients point at a local OpenAI-compatible proxy, which injects memory into the system prompt, forwards the request to the configured provider (cloud or local), and captures the conversation for memory formation; Claude Desktop connects through an MCP server and the browser extension through a WebSocket. These routes do not behave alike, and the experiments use only the proxy (§4).

### 3.1 Thought taxonomy and two-phase capture

Conversational memory is typed into seven categories with distinct persistence semantics (Table 1). An *identity* thought ("I am a backend engineer") should outlive an *event* ("I tried Bun yesterday"), and the taxonomy encodes this as type-specific half-lives.

Capture is two-phase. *Phase 1* is synchronous: embed, deduplicate by hash, insert as *tentative* (median 21 ms, p99 684 ms over 100 adds on an x86-64 Mac). *Phase 2* runs in the background, so the conversation never waits for it; building the injected context adds a median of 21 ms (p99 55 ms) per request. In *Phase 2*, an extractor (a small cloud LLM, or a deterministic fallback) classifies the thought; a confidence scorer updates its status; a supersession check retires contradicted predecessors; the cold store (Markdown + YAML frontmatter in an Obsidian vault) is updated for human readability and editability.

Table 1: Thought taxonomy with type-specific half-lives and injection priority.

| **Type** | **Definition** | **Half-life (days)** | **Priority** |
|---|---|---|---|
| identity | Who the user is | $\infty$ | highest (always) |
| preference | Explicit preference | 90 | high |
| position | Stance on a topic | 90 | high |
| question | Open question under exploration | 30 | medium |
| synthesis | Insight from merged ideas | 180 | medium |
| event | Happened / decided | 7 | low (recent) |
| contradiction | Self-contradictory judgment | $\infty$ | flagged |

### 3.2 Confidence lifecycle

Every thought begins *tentative*: stored, but never injected and invisible to retrieval—the "capture threshold low, injection threshold high" principle. Four signals promote a thought: *repetition* (the same stance recurs across sessions), *assertion strength* ("I've decided" outranks "maybe"), *cross-tool consistency* (distinct phrasings of a stance from different tools), and *absence of contradiction*. The scorer combines them as $0.35\times$repetition$+0.25\times$assertion$+0.25\times$no-contradiction$+0.15\times$cross-tool, each scored from 0 to 1; a thought becomes provisional at 0.45 and settled at 0.75. A single firmly stated thought with no conflict ("I prefer X", assertion 0.8) therefore scores exactly 0.45 and becomes injectable, while settling always requires repetition: at least three similar earlier thoughts with cross-tool corroboration, or six without. *Provisional* thoughts inject when topic-relevant; *settled* thoughts always inject. The result is a rule-based admission score: it decides what may be injected, not whether a thought is true, and a score of 0.75 is not a calibrated 75% probability of being correct [7]. Confidence evolves incrementally: reinforcement $+0.10\,(1 - C)$, conflict $-0.20$, and time decay $C \leftarrow C\,e^{-0.693\,\Delta t/t_{1/2}}$ with $t_{1/2}$ set by type (Table 1).

### 3.3 Cluster-aware supersession

When a user changes their mind ("I moved from MySQL to PostgreSQL"), the old stance should retire. Embedding similarity is the wrong signal here: contradictory stances mention different entities and therefore have *low* similarity. Noesis detects mind-changes by requiring jointly (1) a replacement verb ("switched to", "migrated to", …, EN+ZH), (2) the same domain via a keyword map (both stances are about *databases*), (3) stance-like type, and (4) injectable status of the new

thought. Matched predecessors are marked *superseded*—excluded by every retrieval path—and linked via `superseded_by`. In my evaluation (the same 30 scenarios, run once before and once after the fix) this reduced stale-stance leakage from 93% to 7% (Table 2).

Table 2: Supersession fix: stale-stance leakage before/after ($n$=30 scenarios).

| Metric | Before | After | Δ |
|---|---|---|---|
| Old stance leaked into context | 28/30 (93%) | 2/30 (7%) | −87pp |
| New stance surfaced | 12/30 (40%) | 28/30 (93%) | +53pp |
| Ideal (new shown, old retired) | 1/30 (3%) | 27/30 (90%) | +87pp |

### 3.4 Selective retrieval and injection

At query time, three channels run in parallel (Figure 2): BM25 for exact terms, dense retrieval for paraphrase, and two lightweight signals (recency, core-facts). Channels fuse via reciprocal-rank fusion with time-decay weighting. The semantic channel also attempts a one-hop expansion over links between notes in the vault; in the current code this step adds nothing (§7), so every reported result comes from flat retrieval. A confidence gate drops everything below *provisional*, and a hard token budget (1,200 tokens) truncates the remainder. The result is injected into the system prompt as compact structured lines.

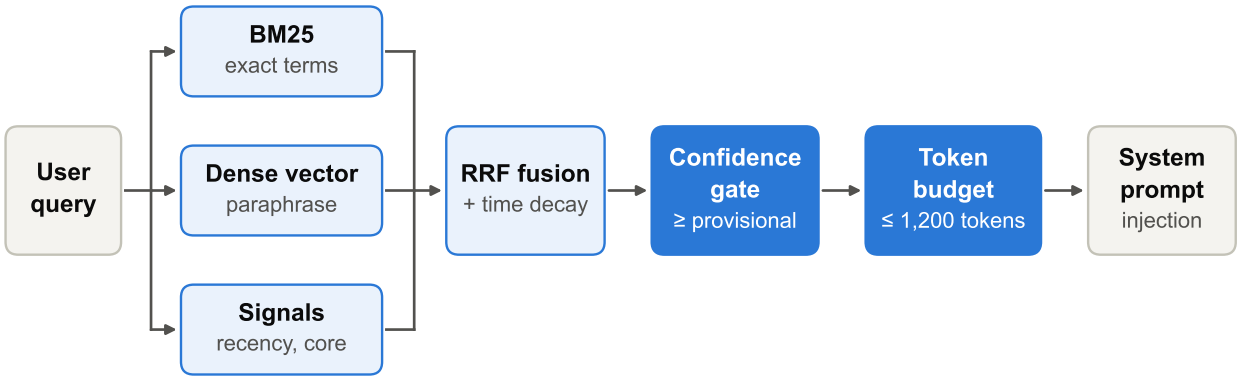



Figure 2: Retrieval and injection pipeline. Three channels fuse via RRF; a confidence gate removes non-injectable thoughts; a token budget caps injected context.

## 4 Evaluation Setup

I evaluate four research questions:

- **RQ1**: Does memory injection raise the keyword-hit rate on personal questions? (A/B, end-to-end, §5.1)
- **RQ2**: Which components contribute? (ablation, retrieval-layer, §5.2)
- **RQ3**: How do cost and ranking scale with memory size? (§5.3)
- **RQ4**: Does cross-tool sharing work? (§5.4)

**Models.** Gemini Flash-Lite (cloud; `gemini-flash-lite-latest`, queried 30 June 2026), gemma3:4b and qwen2.5:3b (local via Ollama), spanning capability tiers.

**A/B design.** 30 synthetic English user profiles, each with 5 build statements (an identity statement plus preferences, positions, and decisions) followed by 5 test questions whose expected keyword is known from the build statements. Each build statement is stated once and firmly ("I prefer…", "I decided…"), which is enough to reach provisional (§3.2). For each question the model answers twice: *with memory* (via the Noesis proxy, which injects context) and *without memory* (direct to the provider). All models are evaluated on the identical first 110 questions (22 profiles). An answer counts as a hit if it contains the expected keyword, and I report the keyword-hit rate; paired outcomes are tested with McNemar's $\chi^2$.

**Retrieval-layer experiments.** Ablation, scale, and cross-tool experiments run against the real pipeline without LLM calls, measuring Recall@5, MRR, the recall and size of the full injected context, tentative leakage, and injected tokens. They call the Python API directly, bypassing all three integration routes; the cross-tool experiment simulates different tools by labeling each stored statement with a source tool rather than by running real clients.

**Integration routes.** Noesis offers three routes into the same store (Figure 1), but they behave differently, and this paper evaluates one of them end to end. The *proxy* retrieves context for the latest user message, appends it to the system message, forwards the request, and stores the exchange, so injection and capture are automatic; the A/B and end-to-end experiments (§5.1) run through it. The *MCP server* exposes five tools (`remember`, `recall`, `wiki_query`, `inspect_memory`, `memory_status`). Under MCP the host coordinates the model and aggregates context, and tools are model-controlled [18]: nothing is stored unless the model calls `remember`, in words the model chooses, and nothing is injected unless it calls `recall`. Exposing these tools therefore does not ensure that they are called at the right time, and the MCP route is not equivalent to the proxy. The server implements the 2024-11-05 protocol revision and has unit tests, but no experiment here uses it. The *browser extension* sends each completed user–assistant turn on the ChatGPT, Claude, Gemini, and Perplexity web apps to the daemon over a WebSocket. It only captures: it cannot inject memory into those pages, and it is not evaluated here.

**Reproducibility.** Every experimental number in this paper is recomputed from the raw result files by a released script (`experiments/verify_paper.py`), which also reads the lifecycle constants from the source. The ablation and scale experiments were re-run in October 2026 with the June software versions; they reproduced the June numbers exactly before the corrected leak metric and the top-5 baseline were added. The ablation was later corrected further (§9). `verify_paper.py` is an arithmetic audit: it checks that each reported number follows from the saved per-question records, not that each answer was scored correctly. Those records are incomplete. The local-model A/B runs kept the first 150 characters of each answer and the end-to-end run the first 100, although hits were scored on the full answer; the Gemini A/B run kept no answer text, because its records were recovered from the run's console log, which printed only hit marks; and no run kept the full prompt or the injected context. The released files therefore support checking the arithmetic, not re-scoring the answers. The code, data, and paper sources used here are tagged `v1.0-preprint` in the repository.

# 5 Results

## 5.1 RQ1: Memory injection raises the keyword-hit rate across model tiers

Table 3: A/B keyword-hit rates on identical $n$=110 questions. All $p < 0.001$ (McNemar, df=1).

| Model | $n$ | With | Without | $\Delta$pp | $\chi^2$ |
|---|---|---|---|---|---|
| Gemini Flash-Lite | 110 | 85.5% | 41.8% | **+43.6** | 42.48 |
| gemma3:4b | 110 | 82.7% | 34.5% | **+48.2** | 51.02 |
| qwen2.5:3b | 110 | 62.7% | 37.3% | **+25.5** | 16.57 |

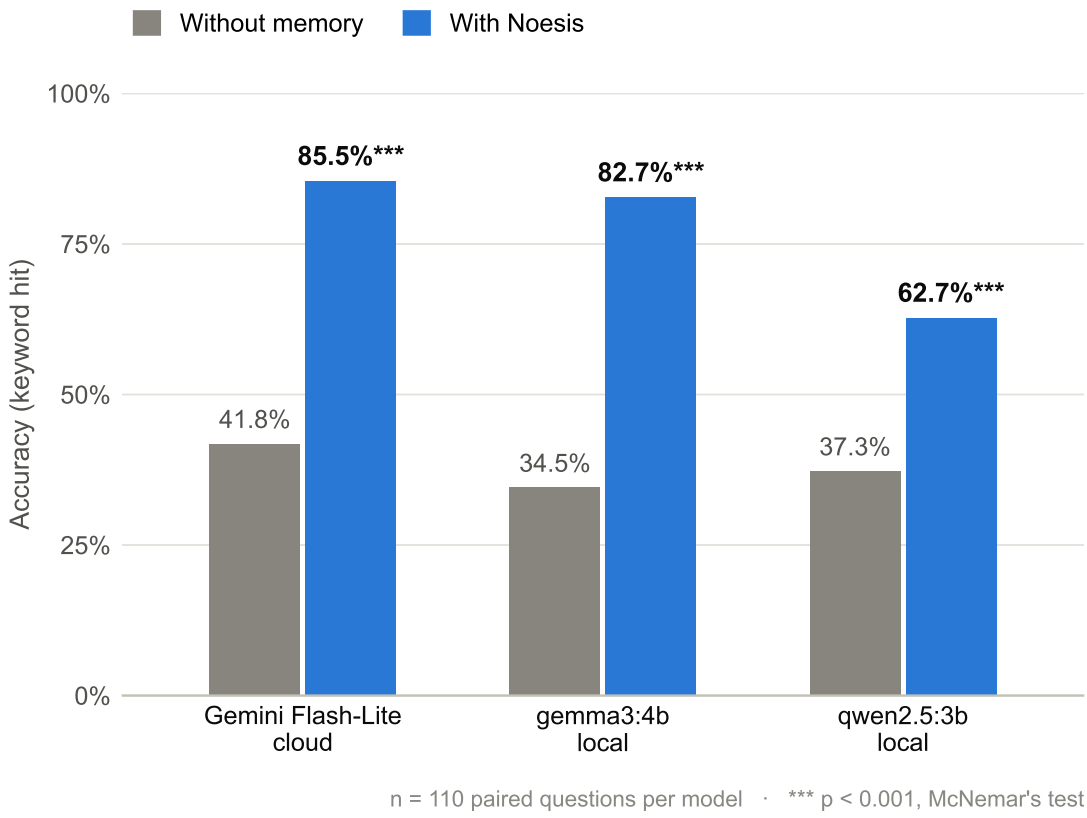



Figure 3: A/B keyword-hit rate across models ($n$=110 each). All improvements significant at $p < 0.001$.

Table 3 and Figure 3 show the headline result. Three findings stand out. First, *injection helps everywhere*: all three models improve significantly, and memory *hurt* in only 10 of 330 paired answers (3.0%). Second, *memory levels the field between local and cloud*: with injection, gemma3:4b (4B parameters, local) reaches 82.7%, nearly matching Gemini Flash-Lite's 85.5%, while its no-memory baseline (34.5%) trails Gemini's 41.8%. Third, *small models appear comprehension-capped*: qwen2.5:3b goes through the same retrieval pipeline but reaches only 62.7%, which points to a model-capability ceiling rather than a retrieval failure; the saved runs do not record the injected context, so this is not yet confirmed.

The hit rate alone does not separate retrieval strategies at this scale. In a separate end-to-end run on 50 questions (10 profiles, Gemini Flash-Lite), Noesis, injecting every stored memory, and injecting the five most recent memories scored 41, 40, and 41 of 50, against 22 of 50 with no memory. With only 3–9 stored memories per profile in that run, all three strategies inject nearly the same context; they diverge on filtering and cost as the store grows (§5.2–§5.3). The A/B result therefore shows that having personal context available helps on these tasks; it does not show that the confidence lifecycle beats simpler ways of supplying the same memories, which the end-to-end run could not separate.

### 5.2 RQ2: Semantic retrieval drives recall; gating keeps tentative thoughts out

Table 4: Ablation and baselines (50 queries over 28 memories, 5 of them tentative; retrieval-layer, 2,000-token budget). A retrieved memory counts only if it is the ground-truth memory itself. Recall@5 and MRR use the first five memories in injection order, and a query that returns fewer than five memories is scored on what it returns; context recall, items, and leak cover everything the configuration injects, and leak counts tentative memories among those slots.

| Config | Recall@5 | MRR | Context recall | Items | Leak |
|---|---|---|---|---|---|
| **Noesis (full)** | **99%** | **1.000** | **100%** | **10.9** | **0/546** |
| w/o core-fact signal | 99% | 1.000 | 100% | 9.7 | 0/487 |
| w/o recency signal | 99% | 1.000 | 100% | 9.7 | 0/484 |
| w/o confidence gating | 99% | 1.000 | 100% | 11.5 | **73/573** |
| w/o semantic retrieval | 24% | 0.105 | 24% | 5.0 | 0/250 |
| Top-5 RAG | 99% | 1.000 | 99% | 5.0 | 19/250 |
| Recent-window | 17% | 0.084 | 17% | 5.0 | 50/250 |
| Random | 12% | 0.063 | 12% | 5.0 | 51/250 |

Each Noesis variant runs the real context builder with exactly one component switched off, so signal order, deduplication, and the token budget stay the same; the baselines draw from every stored memory, tentative ones included. Status and confidence are assigned when the test store is built, so this suite tests whether retrieval respects those labels, not whether the lifecycle derives them from conversation. With 28 memories the semantic channel is dense retrieval alone (BM25 fusion starts at 300 memories).

Removing semantic retrieval collapses recall (99%→24%), confirming it as the main source of recall (Table 4). Removing the confidence gate leaves recall unchanged but lets tentative thoughts into 73 of the 573 injected slots, where the full system injects none—gating is what keeps *tentative* material out of the prompt. Ungated top-5 retrieval ranks as well as Noesis here (Recall@5 99%, MRR 1.000), as it does at scale (§5.3), but injects a tentative thought in 19 of its 250 slots. A separate poisoning test gives the same picture: with 5 settled facts and 8 hedged statements that contradict them, ungated top-5 retrieval returned a hedged statement in 13 of 25 slots across 5 queries, and Noesis in none. Both tests use hedged statements; a firmly stated false one would pass the gate (§9). Removing the core-fact or recency signal changes nothing measurable: each adds about one memory per query (10.9 vs. 9.7 items) without changing recall. The builder adds these signals after the eight semantic results, so they never reach the first five slots; whether they help on queries the semantic channel misses needs a dedicated query set.

### 5.3 RQ3: Retrieval keeps cost flat as memory grows

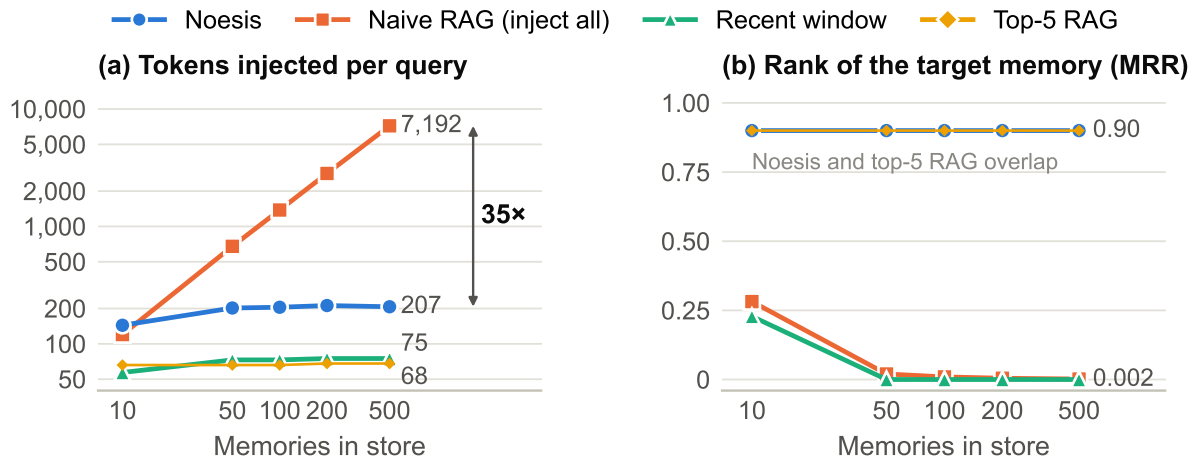



Figure 4: Scale behavior from 10 to 500 stored memories. (a) Noesis's injected tokens stay flat (∼207) while naive RAG (inject all) grows to 7,192 (35 times); top-5 RAG stays at 66–68. (b) Noesis and top-5 RAG hold MRR at 0.90; naive RAG collapses to 0.002 as targets are buried; recent-window loses recall entirely past $n$=10.

Figure 4 sweeps memory size from 10 to 500 with 10 known targets embedded among fillers. Noesis's injected cost is flat (144→207 tokens) because retrieval returns a small, confidence-gated set; the 1,200-token budget is never reached. MRR holds at 0.90: 9 of the 10 targets sit at rank 1, and the tenth is never retrieved. Naive RAG keeps recall only by injecting everything: 7,192 tokens at $n$=500, with the target buried near rank 500 (MRR 0.002). Recent-window recall collapses to 0% beyond $n$=10: the window forgets everything older. A standard top-5 RAG, which embeds the query and injects its five nearest memories, does as well as Noesis on this benchmark: the same 9 of 10 targets at rank 1, with 66–68 tokens per query. Flat cost and good ranking therefore come from retrieval itself rather than from anything specific to Noesis; what Noesis adds is the gate (§5.2), which top-5 RAG lacks. The methodological takeaway: *recall alone is misleading*; rank and token cost must be reported together.

### 5.4 RQ4: Cross-tool sharing vs. structural isolation

Of 40 scenarios, 27 state a stance in one tool and ask about it from another; in 7, the asking tool was one of two tools that stated it; and in 6, a third tool asks after two others stated it (Figure 5). On the 27 strictly cross-tool queries, Noesis retrieves 37% (10/27) and the isolated baseline 0%—not because it ranks poorly, but because the memory is not in its partition, so that 0% holds by construction. The 7 scenarios where the asking tool did see the stance are a fairer comparison: isolated memory retrieves 3 of 7 and Noesis 6 of 7. Over all 40 scenarios, Noesis retrieves 50% and the isolated baseline 7.5%. This is the structural argument: per-tool memory systems cannot share by design, and no amount of per-tool tuning closes the gap. The comparison therefore shows the value of access to a shared store, not an advantage over another shared-memory system, which would need both systems to see the same records. Noesis's misses trace to retrieval precision (the memory exists but ranks below top-$k$), not to sharing.

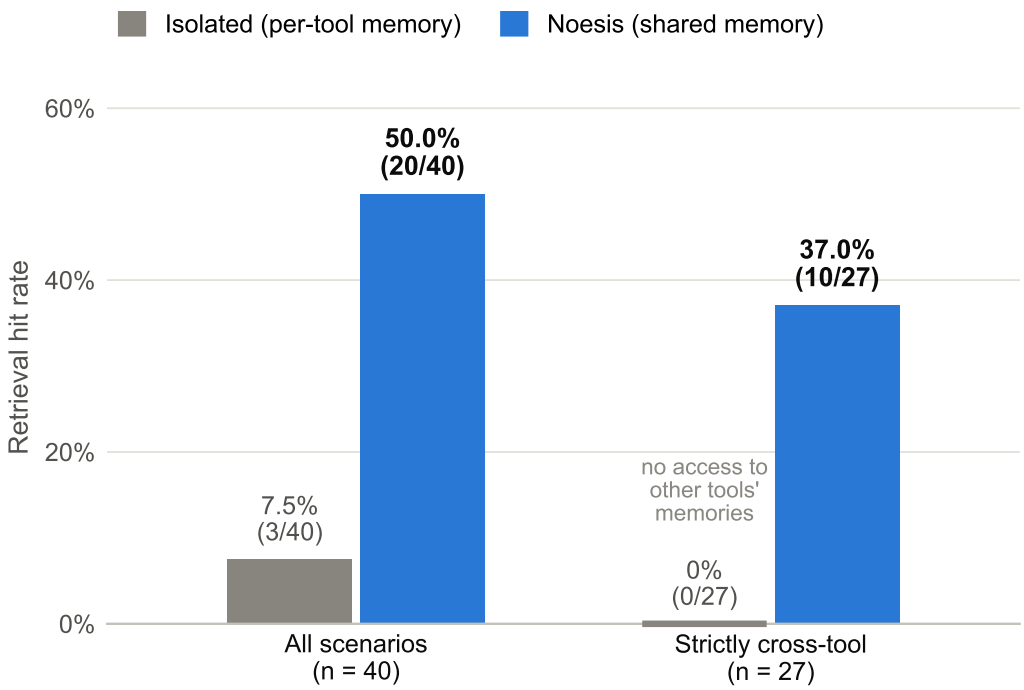



Figure 5: Cross-tool retrieval. Isolated per-tool memory scores 0% on strictly cross-tool queries by construction.

## 6 The Star Map Visualization

The star map displays the stored thoughts as a galaxy (Figure 6). Each thought is a star placed by a short force simulation. Weak springs pull each star toward its creation time on the horizontal axis and its confidence on the vertical axis; *semantic attraction* pulls it toward its most similar neighbors (embedding cosine $\geq 0.45$); and *repulsion* keeps stars from overlapping while pushing different topics apart. The springs are weak enough that attraction wins locally, so clusters form from embedding similarity while time and confidence stay readable at a glance; topic labels only decide which clusters repel. This makes the memory store inspectable rather than a black box: older thoughts sit to the left, settled ones sit high, and hovering a star shows its text and confidence. The store shown is the local development store, populated by the experiment and demo runs rather than by real use: of its 601 thoughts, 355 sit in the catch-all "general" topic, which the extractor assigns when no known topic matches, and 18 have settled. Even so, the view surfaces such patterns at a glance; here it shows how coarse topic assignment still is. The viewer is a local web tool shipped with the system; it opens the store read-only, so inspecting memory never changes it. The map is a display, not a retrieval structure: its attraction edges are computed by the viewer, are never stored, and do not affect what is retrieved or injected, so it says nothing about whether graph structure improves answers.

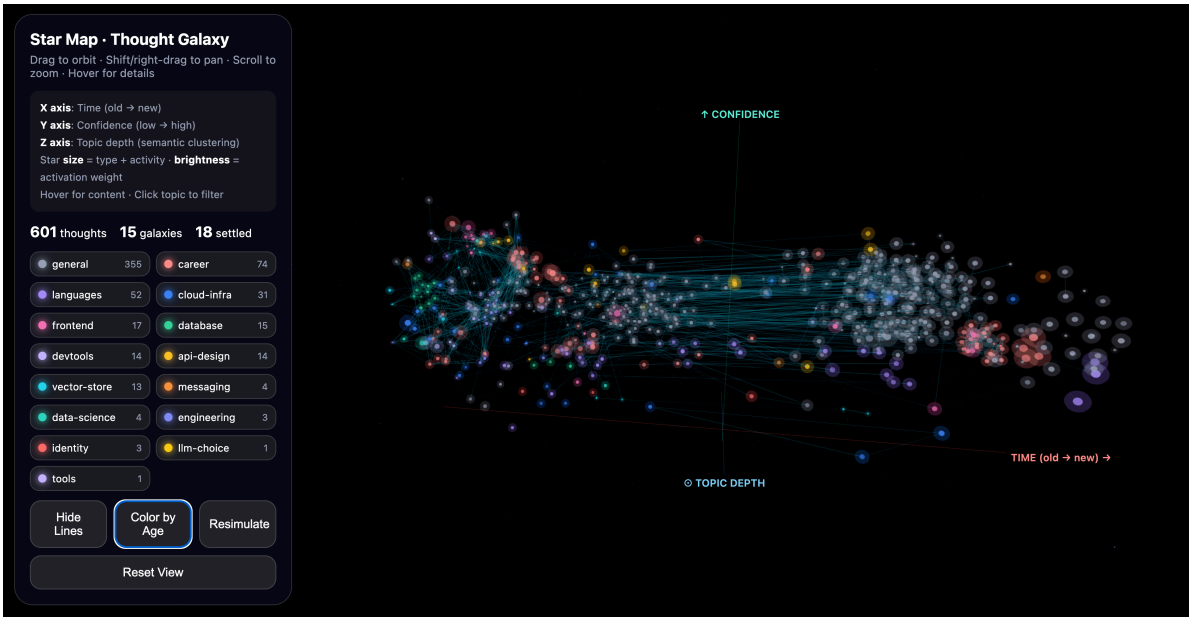



Figure 6: The star map. Semantic neighbors attract into volumetric constellations. Time runs from older (left) to newer (right), height reflects confidence, and color encodes topic; star size encodes type and activation weight, and brightness also tracks activation weight.

# 7 Discussion

**When memory helps most.** Gains concentrate on questions that are unanswerable without personal context (job, stack, stances). On mainstream questions both groups often answer correctly, which compresses the measured gap on those items. Other factors push the other way (see Limitations), so these results should not be read as a bound in either direction.

**Memory can close the local–cloud gap.** gemma3:4b posts the largest gain (+48.2pp; +139% over its baseline), approaching Gemini Flash-Lite once memory is injected. The benefit is not monotonic in model size: qwen2.5:3b gains least (+25.5pp), consistent with the comprehension ceiling in §5.1. Memory helps small local models most when they can use the injected context, which matters for privacy-sensitive and offline settings.

**Retrieval precision is the current bottleneck.** The cross-tool misses and several A/B misses reduce to ranking failures, not architecture failures: the memory is stored but ranks below the cut. Improving ranking (rerankers, query expansion) is the highest-leverage next step. The supersession fix's residual 7% is different: in both remaining leaks the detector never fired, once because "I now believe microservices are better" uses no listed replacement verb and names no known domain, and once because "I switched to Node.js" has the verb but Node.js is missing from the domain map. A broader mind-change detector is the matching fix.

**Noesis inside an agent harness.** An execution harness is the software around a model that manages a task's control flow, tool use, state, and recovery, for example by carrying progress across sessions [1]; it is distinct from the experiment harness used in this paper. Noesis fits inside such a harness as a memory service, not as a replacement for orchestration, and what follows is a proposal

that this paper does not implement or evaluate. Before a model step, the harness would identify the user, project, task, and access scope, request approved memories, and insert a bounded evidence block that names each memory's source; after the step, it would send candidate memories with their sources for asynchronous processing; and it would keep an explicit `recall` call for when a tool result changes the question. LangGraph shows the intended division of labor: it keeps thread-scoped checkpoints of a run's state separate from a cross-thread store for long-term memory such as user preferences [11]. Noesis could fill the second role but not the first: remembering that a user prefers PostgreSQL is different from recording which step of an interrupted database migration has finished.

**What a harness integration would need.** A harness would need a contract that the current code does not provide: reads, candidate writes, corrections, deletion, and audit, each with stable event identifiers and a record of where every memory came from. Today, corrections and deletions happen only by editing or deleting the Markdown files in the vault. Three further gaps are visible in the code. First, the repetition signal counts every stored record in the same topic that shares at least two keywords, whatever its source, and the proxy stores each request's full message list; replayed history, retried requests, and assistant paraphrases can therefore raise a memory's confidence as if new evidence had arrived. I have not measured how often this happens. Second, a statement is stored at once but becomes injectable only after the asynchronous Phase 2 scores it, and read consistency under concurrent agents is unspecified. Third, retrieved memories are appended to the system message under a plain heading, in the same message as the host's own instructions; an integration should mark them as data, deduplicate them against the harness's own summaries and instruction files, and never let them grant permissions or override higher-priority instructions.

**Graph structure: what exists and what is tested.** Noesis keeps several kinds of links, but the evaluated system is not a graph-based reasoner. Supersession links (`superseded_by`) are implemented and evaluated (Table 2). Topic clusters are implemented and used, since the scorer counts repetition and contradiction only within a thought's topic, but no experiment isolates their effect. Links from thoughts to wiki pages (`fact_ref`) are implemented but not evaluated. The semantic channel also calls a one-hop expansion over the *Related* links between notes in the vault, but in the current code it cannot add a memory: links between thoughts are written with 8-character identifiers, while the store looks up full 12-character identifiers, and topic and wiki links do not point to stored thoughts. The ablation and scale stores were also built without the background pipeline, so they contain no such links. Every reported retrieval result therefore comes from flat retrieval. The star map (§6) is a visualization only, and typed claim graphs, branching, and structured injection are proposals (§8).

**Negative results.** I report that the supersession check initially never ran, so stale stances leaked 93% of the time until it was fixed (§3.3); that core-fact and recency signals added no recall in my ablation suite; that a standard top-5 RAG matches Noesis's ranking at lower cost on the scale benchmark; that a 3B model appears comprehension-capped; and that the one-hop graph expansion in the retrieval path never adds a memory. I consider these findings part of the contribution.

## 8 Future Work

**Thought branching.** When a new judgment deviates from history by more than a threshold ($\delta = 1 - \cos > 0.45$), Noesis should create a *branch* linked via `evolved_from` rather than superseding,

preserving the full trajectory of a stance. This enables metrics I cannot compute today: stabilization depth, branching rate, and post-branch divergence—a longitudinal record of how a user's positions evolve.

**Structured injection.** Replacing flat context lines with a structured `user_thought_graph` (anchor topic, trajectory with weights, cross-links) may improve the ability of small models to exploit injected memory, directly addressing the qwen2.5 comprehension ceiling.

**Stronger evaluation.** LLM-as-judge scoring to complement keyword hits; distractor questions; a second cloud model to replicate the Gemini result; longitudinal deployment data beyond synthetic profiles.

**Harness-level evaluation.** With the model, tasks, tools, and budgets held fixed, compare a harness's native memory, a simple shared store, and Noesis-backed memory on task completion, correct use of preferences, resumption after an interruption, stale-information errors, isolation between projects, and unnecessary retrieval calls, counting the full cost of capture, extraction, maintenance, retrieval, and generation rather than injected tokens alone. Counting repetition only across distinct user statements, with their sources recorded, is the matching fix for the first gap in §7.

**Typed temporal claim graph.** A remembered claim could link to a user, project, entity, source utterance, tool, and time interval, with explicit relations for support, contradiction, derivation, and supersession; project and time scope would let "PostgreSQL for analytics" and "MySQL for a legacy application" coexist instead of conflicting. Zep's temporal graph is the closest precedent [21]. Derivation links matter for provenance: ten claims derived from one utterance are not ten observations, and a frequently copied error can become highly connected, so degree or centrality should not raise confidence. The deciding experiment crosses representation with admission policy: a flat store and a graph-backed store, each with and without the same confidence gate, with extraction, source access, backbone model, and token budget held fixed, reporting retrieval and answer correctness alongside graph-construction cost, latency, the rate of incorrect links, and the effect of removing or correcting a source. Fixing the identifier mismatch above would let the existing one-hop expansion serve as a first, simple graph condition. Learned graph models are a further step that would need held-out users, domains, and time periods.

## 9 Limitations

Scoring is keyword-hit, which is objective but coarse: it checks whether an answer contains the expected keyword, not whether the answer is right. "You do not prefer Python" contains the same keyword as "you prefer Python", and a correct answer worded differently or given in another language counts as a miss. Personas are synthetic; build/test alignment may overstate real-world gains, and real conversations are noisier. The cloud A/B uses one provider (Gemini Flash-Lite) and $n$=110 rather than the planned 150: free-tier rate limits stopped the 30 June run after 111 questions. A later session added 35 more, but because the `gemini-flash-lite-latest` alias may have pointed to a different model version by then, I use only the first 110 questions (22 complete profiles) from the single June run; all significance thresholds are still exceeded by wide margins. Retrieval-layer experiments measure what reaches the prompt, not final answer quality. The supersession detector relies on replacement verbs and a domain keyword map; paraphrased mind-changes outside its lexicon will be missed. The supersession fix was developed and validated on the same 30 scenarios,

so its 93% → 7% result is in-sample; held-out, paraphrased mind-changes are needed to measure it fairly. The scale experiment's naive-RAG baseline injects every stored memory; the top-5 baseline added in §5.3 is the stronger comparison, and on this benchmark it ties Noesis on ranking. The ablation and poisoning leak counts were corrected in October 2026: the original keyword-based metric also counted settled memories that merely shared a word with a tentative one (for example, "Kafka"). After an external review, I corrected the ablation further: it now scores exact memory identity (the old phrase matcher credited any memory sharing wording such as "I prefer"); it drops "Precision@5", which counted non-tentative results rather than relevant ones; it runs every variant through the same context builder (the old core-fact and recency rows could not differ from the full system); it applies the memories' ages, which the test store had silently dropped; and its recent-window and random baselines can now return tentative memories. Together these changes lowered the weak baselines (random 45% → 12%, w/o semantic 49% → 24% Recall@5) while Noesis moved from 100% to 99%.

Noesis also adds an attack surface. The local proxy sees every request, including provider API keys and full conversations; enabling cloud extraction sends conversation text to a third-party API, which weakens the local-first guarantee; and because a single firmly stated sentence already reaches provisional (§3.2), a planted or mistaken statement can enter future prompts. The confidence gate filters hedged statements but is not a defense against deliberate poisoning.

## 10 Conclusion

Noesis shows that *what* a memory system injects matters as much as *whether* it injects. A confidence lifecycle decides which thoughts may be injected: in constructed tests its gate kept tentative and hedged statements out of the prompt where plain retrieval let them in, though it judges how firmly something was said, not whether it is true, and is no defense against deliberate poisoning. Retrieval keeps cost flat as memory grows (35 times cheaper than injecting everything at $n$=500, though a standard top-5 RAG is cheaper still); and a shared per-user store makes cross-tool consistency possible where per-tool architectures are structurally bound to fail. The result is a local-first layer that lets a 4B local model reach nearly the keyword-hit rate of a small cloud model (82.7% vs. 85.5% for Gemini Flash-Lite; the gap is not significant on this set, though that does not show the two are equivalent)—and that keeps the user's reasoning in the user's own vault.

## Acknowledgments

AI coding assistants were used in this work. ZCode (Z.ai, GLM models) assisted with implementing the experiment scripts and drafting the original experiment reports. Claude (Anthropic) was used to review the manuscript, revise its text, redesign the figures, write the verification script (`experiments/verify_paper.py`), and re-run the retrieval-layer experiments in October 2026. All reported numbers are checked against the raw results by that script, and the author takes full responsibility for the paper's content.

## References

[1] Anthropic. Effective harnesses for long-running agents. https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents, 2025. Anthropic engineering blog, November 26, 2025.

[2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.11511.

[3] Prateek Chhikara, Dev Khant, Sparsh Aryan, Harshit Singh, Tanmay Yadav, Srijan Chawla, Nitesh Khare, Shiv Muthuraman S, Aman Gupta, Shantam Mehta, Taran Parekh, Priyansh Deshmukh, Devansh Dhamecha, Ashwini Kankrej, Kinar Kotecha, Srikar Revi, MR Bedmutha, Devendra Khanna, and Chintan Modi. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025. ECAI 2025.

[4] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. Memory injection attacks on LLM agents via query-only interaction. *arXiv preprint arXiv:2503.03704*, 2025.

[5] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[6] Thomas M. J. Fruchterman and Edward M. Reingold. Graph drawing by force-directed placement. *Software: Practice and Experience*, 21(11):1129–1164, 1991.

[7] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.

[8] Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, et al. Memory in the age of AI agents. *arXiv preprint arXiv:2512.13564*, 2025. Survey preprint.

[9] Shuo Ji, Yibo Li, and Bryan Hooi. Memory is reconstructed, not retrieved: Graph memory for LLM agents. *arXiv preprint arXiv:2606.06036*, 2026. ICML 2026.

[10] Zheng Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7969–7992, 2023.

[11] LangChain. Persistence. https://docs.langchain.com/oss/python/langgraph/persistence, 2026. LangGraph documentation, accessed October 17, 2026.

[12] Christopher Latimer, Nicolò Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight: Structured agent memory that retains, recalls, and reflects. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 275–285, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-demo.27. Preprint: arXiv:2512.12818.

[13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[14] Zehao Lin, Xixuan Hao, Renyu Fu, Shaobo Cui, Kai Chen, Chunyu Li, Zhiyu Li, and Feiyu Xiong. A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle. *arXiv preprint arXiv:2604.16548*, 2026. EMNLP 2026.

[15] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[16] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.

[17] Mem0. Introducing OpenMemory MCP. https://mem0.ai/blog/introducing-openmemory-mcp, 2025. Vendor announcement, Mem0 blog, May 13, 2025.

[18] Model Context Protocol. Model context protocol specification, revision 2026-07-28: Architecture and tools. https://modelcontextprotocol.io/specification/2026-07-28/architecture, 2026. Specification, accessed October 17, 2026.

[19] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[20] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[21] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

[22] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008.

[23] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025.

[24] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[25] Chang Yang, Chuang Zhou, Yilin Xiao, Su Dong, Luyao Zhuang, et al. Graph-based agent memory: Taxonomy, techniques, and applications. *arXiv preprint arXiv:2602.05665*, 2026. Survey preprint.

[26] Liangwei Yang, Zhiwei Chen, Lixin Xia, Chen Yang, Yu Zhang, and Ben Kao. PALR: Personalization aware LLMs for recommendation. *arXiv preprint arXiv:2305.07622*, 2023.

[27] Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. *arXiv preprint arXiv:2601.01885*, 2026. ACL 2026.

[28] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.